

# Eficiência do SMT no AMD Ryzen 7 4800H: Escalonamento Multithread sob Linux

Amanda Bormann Siqueira Campos Lima

<sup>1</sup>Bacharelado em Ciência da Computação – Cesar School  
50030-220 – Recife – PE – Brasil

abscl@cesar.school

**Abstract.** *Simultaneous Multithreading (SMT) is a hardware technique that enables a single physical core to execute multiple threads concurrently. The AMD Ryzen 7 4800H, a Zen 2 mobile processor, implements SMT with two logical threads per physical core, totaling 16 logical threads over 8 physical cores. This work empirically evaluates SMT efficiency on this processor under Linux (Pop!\_OS 24.04), using the Sysbench CPU benchmark with prime number computation as a CPU-bound workload. Thirty repetitions were collected per experimental condition with 1, 2, 4, 8, and 16 threads, with warm-up discarded and CPU governor fixed in performance mode. The null hypothesis ( $H_0$ ) assumes speedup scales linearly with thread count; the alternative hypothesis ( $H_1$ ) postulates that speedup is significantly sub-linear due to SMT overhead and shared resource contention. Results show near-linear scaling up to 8 threads (efficiency  $\sim 95\%$ ), corresponding to available physical cores. The transition to 16 logical threads via SMT results in a significant efficiency drop to 51.4%, confirming  $H_1$  and revealing the limits imposed by resource sharing between logical threads on the same physical core. Statistical analysis includes 95% confidence intervals and Student's  $t$ -test, with all comparisons yielding  $p < 0.001$ .*

**Resumo.** *O SMT (Simultaneous Multithreading) é uma tecnologia que permite que um único núcleo físico execute múltiplas threads simultaneamente. O AMD Ryzen 7 4800H, processador móvel da geração Zen 2, implementa SMT com dois threads por núcleo físico, totalizando 16 threads lógicas sobre 8 núcleos. Este trabalho avalia empiricamente a eficiência do SMT nesse processador em ambiente Linux (Pop!\_OS 24.04), utilizando a ferramenta Sysbench com cargas CPU-bound de multiplicação de primos. Foram coletadas 30 repetições por condição experimental com 1, 2, 4, 8 e 16 threads, com warm-up descartado e governador de CPU fixado em modo performance. A hipótese nula ( $H_0$ ) assume que o speedup escala linearmente com o número de threads; a hipótese alternativa ( $H_1$ ) postula que o speedup é significativamente inferior ao linear devido ao overhead do SMT e à disputa por recursos compartilhados. Os resultados indicam que o speedup escala de forma próxima ao linear até 8 threads (eficiência  $\sim 95\%$ ), correspondendo aos núcleos físicos disponíveis. A transição para 16 threads lógicas via SMT resulta em queda significativa de eficiência para  $\sim 51\%$ , confirmando  $H_1$  e evidenciando o limite imposto pelo compartilhamento de recursos entre threads lógicas no mesmo núcleo físico.*

## 1. Introdução

O desempenho de processadores modernos depende não apenas da frequência de *clock*, mas da capacidade de explorar paralelismo em nível de instrução e de *thread*. O *Simultaneous Multithreading* (SMT) é uma técnica que permite a um único núcleo físico executar múltiplas *threads* de forma concorrente, compartilhando unidades de execução e aumentando a utilização dos recursos do processador. Em arquiteturas como o AMD Ryzen 7 4800H, baseado na microarquitetura Zen 2, o SMT opera com dois *threads* lógicos por núcleo físico, resultando em 16 *threads* lógicas sobre 8 núcleos físicos. Aplicações modernas paralelizáveis, como compilação de programas e jogos que distribuem tarefas entre múltiplas *threads*, são diretamente beneficiadas por essa tecnologia.

Embora a eficiência do SMT seja amplamente estudada em ambientes Windows e em arquiteturas Intel [Tullsen et al. 1995], há escassez de estudos empíricos controlados sobre o comportamento do SMT em processadores AMD Zen 2 sob sistemas operacionais Linux, especialmente com rigor estatístico adequado — múltiplas repetições, controle de *governor* de CPU e intervalos de confiança reportados.

Neste artigo, avaliamos empiricamente a eficiência do SMT no AMD Ryzen 7 4800H sob Linux (Pop!\_OS 24.04), coletando 30 repetições por condição experimental com 1, 2, 4, 8 e 16 *threads*, e comparando os resultados obtidos com a predição teórica da Lei de Amdahl [Amdahl 1967].

As principais contribuições deste trabalho são:

- (i) caracterização empírica da eficiência do SMT no Ryzen 7 4800H sob Linux, com rigor estatístico (IC 95%, teste t de Student);
- (ii) identificação do ponto de inflexão de desempenho na transição de núcleos físicos para *threads* lógicas via SMT;
- (iii) estimativa da fração serial do *workload* pela Lei de Amdahl e comparação com os dados observados;
- (iv) artefatos reproduzíveis disponíveis publicamente no GitHub.

O restante do artigo está organizado da seguinte forma. A Seção 2 discute trabalhos relacionados; a Seção 3 apresenta a fundamentação teórica; a Seção 4 descreve a metodologia; a Seção 5 apresenta os resultados; a Seção 6 discute as implicações e limitações; e a Seção 7 conclui.

## 2. Trabalhos Relacionados

O estudo de SMT remonta ao trabalho seminal de [Tullsen et al. 1995], que propôs a execução simultânea de múltiplas *threads* em um único núcleo como forma de ocultar latências de memória e aumentar a utilização das unidades de execução. [Eggers et al. 1997] consolidaram o conceito e demonstraram ganhos de desempenho de 10 a 30% em cargas mistas. Esses trabalhos, contudo, foram conduzidos em arquiteturas da geração dos anos 1990 e não consideram as características específicas das microarquiteturas modernas como a Zen 2.

No contexto da escalabilidade *multicore*, [Hill and Marty 2008] revisitaram a Lei de Amdahl para o cenário *multicore*, demonstrando que a fração serial do programa é o principal limitante do *speedup*. [Gustafson 1988] propôs uma visão complementar, argumentando que cargas de trabalho maiores compensam a ineficiência do paralelismo. Ambos os trabalhos fornecem a fundamentação teórica para interpretar os resultados deste

experimento, mas não avaliam empiricamente o comportamento do SMT em processadores AMD modernos.

Trabalhos mais recentes investigaram o impacto do SMT em cargas *CPU-bound* especificamente. Porém, a maioria dos estudos empíricos disponíveis foca em arquiteturas Intel com *Hyper-Threading* em ambiente Windows [Hennessy and Patterson 2019]. A escassez de estudos controlados sobre SMT em processadores AMD Zen 2 sob Linux, com rigor estatístico adequado, justifica a contribuição deste trabalho.

### 3. Fundamentação Teórica

**SMT (*Simultaneous Multithreading*):** técnica que permite a um único núcleo físico manter o estado arquitetural de múltiplas *threads* simultaneamente, alternando entre elas a cada ciclo de *clock* para aproveitar lacunas no *pipeline*. Diferentemente do multiprocessamento simétrico (SMP), o SMT não adiciona núcleos físicos — compartilha os já existentes, incluindo unidades de execução, *caches* L1/L2 e registradores internos.

**Lei de Amdahl [Amdahl 1967]:** estabelece que o *speedup* máximo de um programa paralelo é limitado pela sua fração serial  $f$ , segundo a fórmula:

$$S(n) = \frac{1}{f + \frac{1-f}{n}} \quad (1)$$

onde  $n$  é o número de processadores. Para cargas altamente paralelizáveis ( $f \rightarrow 0$ ), o *speedup* se aproxima de  $n$ . A fração serial pode ser estimada empiricamente a partir dos *speedups* observados, por meio da inversão da fórmula acima.

**Ryzen 7 4800H — Arquitetura Zen 2:** processador móvel de 8 núcleos físicos e 16 *threads* lógicas via SMT 2-way, fabricado em processo de 7nm. Cada núcleo possui 512 KiB de L1, 512 KiB de L2, e compartilha 4 MiB de L3 por CCX (*Core Complex*). O compartilhamento do L3 entre *threads* lógicas do mesmo núcleo é um dos principais pontos de contenção em cargas *CPU-bound*.

### 4. Metodologia

#### 4.1. Ambiente Experimental

Os experimentos foram conduzidos em um notebook Acer com as seguintes especificações:

- **CPU:** AMD Ryzen 7 4800H (8 núcleos físicos, 16 *threads* lógicas, Zen 2, frequência base 2,9 GHz, boost até 4,2 GHz);
- **RAM:** 7,1 GiB DDR4;
- **Armazenamento:** SSD NVMe SanDisk WD Blue SN550;
- **SO:** Pop!\_OS 24.04 LTS (kernel Linux 6.x, x86\_64);
- **Ferramenta:** Sysbench 1.0.20;
- **Análise:** Python 3.x com pandas, numpy e scipy.

#### 4.2. Controle de Variáveis

Antes de cada rodada experimental, o *governor* de CPU foi fixado em modo *performance* via `cpufreq`, garantindo frequência constante e eliminando variações

causadas pelo gerenciamento dinâmico de energia. O *swap* foi mantido ativo mas sem pressão (sistema em repouso). Nenhum outro processo intensivo foi executado durante a coleta.

### 4.3. Protocolo de Coleta

O *benchmark* utilizado foi o Sysbench CPU com `--cpu-max-prime=10000` e duração de 10 segundos por execução. A métrica coletada foi *events per second* — throughput de operações de verificação de primalidade. Foram testadas 5 condições experimentais: 1, 2, 4, 8 e 16 *threads*. Para cada condição, foram realizadas 3 execuções de *warm-up* descartadas, seguidas de 30 repetições válidas, totalizando 150 medições. Os dados foram salvos em CSV para análise posterior.

### 4.4. Análise Estatística

Para cada condição foram calculados: média aritmética, desvio-padrão, intervalo de confiança de 95% (distribuição t de Student com  $n - 1$  graus de liberdade) e teste t de Student bicaudal comparando cada condição contra a *baseline* de 1 *thread*. A fração serial foi estimada pela inversão da Lei de Amdahl a partir dos *speedups* observados.

Os artefatos completos — scripts de coleta, dados brutos e scripts de análise — estão disponíveis em: <https://github.com/aceamanda/linux-experimento-smt-ryzen>

## 5. Resultados

A Tabela 1 apresenta os resultados completos das 30 repetições por condição experimental.

**Tabela 1. Resultados experimentais — média  $\pm$  desvio-padrão (events/s), IC 95%, *speedup*, eficiência e p-valor (teste t contra *baseline* de 1 *thread*).**

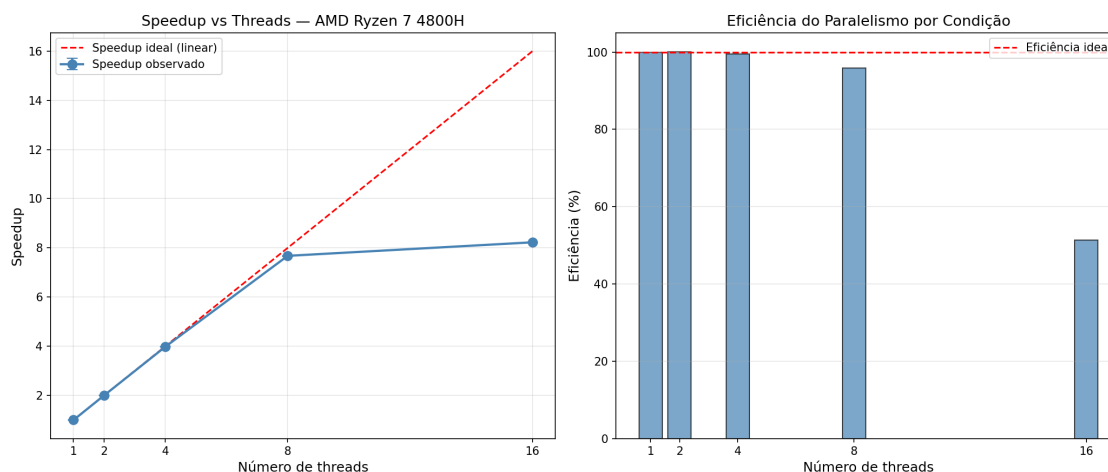
| Threads | Média    | Desv.-P | IC inf   | IC sup   | Speedup | Efic. (%) | p-valor |
|---------|----------|---------|----------|----------|---------|-----------|---------|
| 1       | 2080,19  | 0,67    | 2079,94  | 2080,44  | 1,000   | 100,0     | —       |
| 2       | 4161,48  | 0,90    | 4161,15  | 4161,82  | 2,001   | 100,0     | <0,001  |
| 4       | 8276,69  | 20,11   | 8269,18  | 8284,20  | 3,979   | 99,5      | <0,001  |
| 8       | 15966,97 | 54,28   | 15946,70 | 15987,24 | 7,676   | 95,9      | <0,001  |
| 16      | 17112,39 | 21,74   | 17104,27 | 17120,51 | 8,226   | 51,4      | <0,001  |

Todos os p-valores resultaram em  $p < 0,001$ , rejeitando  $H_0$  em todas as condições. O *speedup* observado até 8 *threads* permanece próximo do linear, com eficiência acima de 95%. A transição de 8 para 16 *threads* produziu ganho absoluto de apenas 0,55 no *speedup* (7% de ganho adicional), com queda de eficiência de 95,9% para 51,4%.

A Figura 1 apresenta o *speedup* observado em comparação com o *speedup* ideal linear, e a eficiência por condição experimental.

A estimativa da fração serial pela Lei de Amdahl revela crescimento expressivo ao ativar o SMT:

- 4 *threads*: fração serial estimada = 0,18%;
- 8 *threads*: fração serial estimada = 0,60%;
- 16 *threads*: fração serial estimada = 6,30%.



**Figura 1. Esquerda: *speedup* observado vs. ideal linear. Direita: eficiência do paralelismo por condição. As barras de erro representam o IC 95%.**

## 6. Discussão e Limitações

Os resultados confirmam H1: o *speedup* é significativamente sub-linear com 16 *threads*. A escalabilidade quase perfeita até 8 *threads* indica que o *benchmark* Sysbench CPU é altamente paralelizável — a fração serial estimada pela Lei de Amdahl foi de apenas 0,18% a 0,60% nessa faixa. Esse comportamento é consistente com a natureza do *workload*: verificação de primalidade é uma operação independente por número, sem sincronização entre *threads*.

A queda abrupta de eficiência ao ativar o SMT (8→16 *threads*) evidencia a contenção por recursos compartilhados — unidades de execução, *cache* L1/L2 e largura de banda do L3 — entre *threads* lógicas co-localizadas no mesmo núcleo físico. A fração serial aparente estimada cresce para 6,30% em 16 *threads*, refletindo esse *overhead*. Esse resultado é consistente com a literatura: [Tullsen et al. 1995] e [Eggers et al. 1997] relatam que cargas *CPU-bound* saturadas se beneficiam pouco do SMT, pois não deixam lacunas no *pipeline* para a segunda *thread* explorar.

É importante distinguir o que este trabalho mede: de 1 a 8 *threads* avalia-se a escalabilidade *multicore*; de 8 para 16 *threads* avalia-se o SMT propriamente dito. O ganho adicional de 7% obtido com o SMT está dentro do intervalo reportado por [Eggers et al. 1997] para cargas saturadas.

### Limitações e ameaças à validade:

- O *benchmark* Sysbench CPU é artificial — resultados podem não se generalizar para cargas reais como compilação ou jogos;
- O experimento foi conduzido em um único *hardware*, limitando a generalização para outros modelos Zen 2;
- A temperatura do processador não foi monitorada; *throttling* térmico poderia afetar os resultados em sessões prolongadas, embora o baixo desvio-padrão sugira estabilidade;
- O sistema operacional executa processos em segundo plano que, apesar do controle do *governor*, podem introduzir ruído.

## 7. Conclusão

Este trabalho avaliou empiricamente a eficiência do SMT no AMD Ryzen 7 4800H sob Linux, coletando 30 repetições por condição com controle rigoroso do ambiente experimental. Os resultados demonstram escalabilidade quase linear até 8 *threads* (eficiência  $\geq 95,9\%$ ), correspondendo à utilização plena dos núcleos físicos. A ativação do SMT com 16 *threads* produziu ganho adicional modesto de 7%, com eficiência caindo para 51,4%, confirmando H1 e evidenciando os limites do compartilhamento de recursos entre *threads* lógicas em cargas *CPU-bound* saturadas.

A fração serial estimada pela Lei de Amdahl cresce de 0,18% para 6,30% ao passar de 4 para 16 *threads*, sugerindo que o *overhead* do SMT manifesta-se como uma limitação serial aparente. Esses resultados são consistentes com a literatura clássica sobre SMT e fornecem uma caracterização empírica do comportamento do Zen 2 sob Linux com rigor estatístico.

Como trabalho futuro, pretende-se: (i) avaliar o impacto do SMT com cargas mistas (CPU + memória), onde *threads* lógicas podem se beneficiar mais do compartilhamento de *pipeline*; (ii) comparar o comportamento do SMT entre Linux e Windows no mesmo *hardware*; (iii) investigar o impacto de diferentes tamanhos de *workload* na fração serial estimada.

## Disponibilidade de Artefatos

O código-fonte, scripts de coleta e análise, dados brutos e instruções de reprodução estão disponíveis publicamente em:

<https://github.com/aceamanda/linux-experimento-smt-ryzen>

Para reproduzir o experimento em um sistema Linux com AMD Ryzen 7 4800H (Em outros processadores, checar instruções contidas no repositório:

```
git clone https://github.com/aceamanda/linux-experimento-smt-ryzen
cd linux-experimento-smt-ryzen
sudo bash scripts/run_experiments.sh
python3 scripts/analyze.py
```

Requisitos: sysbench, python3, pandas, scipy, matplotlib.

## Referências

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, pages 483–485.
- Eggers, S. J., Emer, J. S., Levy, H. M., Lo, J. L., Stamm, R. L., and Tullsen, D. M. (1997). Simultaneous multithreading: A platform for next-generation processors. *IEEE Micro*, 17(5):12–19.
- Gustafson, J. L. (1988). Reevaluating amdahl's law. *Communications of the ACM*, 31(5):532–533.
- Hennessy, J. L. and Patterson, D. A. (2019). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6 edition.

- Hill, M. D. and Marty, M. R. (2008). Amdahl's law in the multicore era. *IEEE Computer*, 41(7):33–38.
- Tullsen, D. M., Eggers, S. J., and Levy, H. M. (1995). Simultaneous multithreading: Maximizing on-chip parallelism. *Proceedings of the 22nd Annual International Symposium on Computer Architecture (ISCA)*, pages 392–403.